

Méthode M5 — régression logistique sur cartes multi-échelles

Challenge 1 · détection des interdits · implémentation dans `detection/baselines.py`

1. Idée

M1 à M4 appliquent *un seul* filtre puis un seuil. Chacun a un défaut propre : le lissage (M1) prend une ligne de charge pour un stick, la courbure (M3) suit la crête des lignes de charge, les bâtonnets (M2) sont corrects mais figés dans une forme. M5 ne choisit pas : elle calcule six cartes complémentaires pour chaque pixel et **apprend la combinaison linéaire** qui sépare au mieux « pixel d'interdit » de « pixel de fond ». Le modèle est un classifieur par pixel à 7 paramètres (6 poids + 1 biais), entraîné sur le masque officiel du générateur. Pas de réseau de neurones, pas de torch : tout est numpy/scipy.

2. Pipeline complet

Image brute (150×150, bruit blanc + rayures horizontales) → **prétraitement** → **6 cartes** → **standardisation** → **logit = $w \cdot x + b$** → **seuil choisi sur val** → masque binaire.

2.1 Prétraitement `prep()` : image en unités de σ du bruit

```
x = x - np.median(x, axis=2, keepdims=True) # (1) retire les rayures (médiane par ligne)
mad = np.median(np.abs(x - np.median(x, axis=(1,2), keepdims=True)), axis=(1,2), keepdims=True)
z = -x / (1.4826 * mad + 1e-6) # (2) signe (3) normalisation MAD
```

(1) Le bruit en rayures est constant le long de l'axe de scan rapide ; les sticks sont rares, donc la médiane d'une ligne est robuste et ne les absorbe pas. (2) Les sticks sont négatifs : on inverse le signe pour qu'un stick soit un pic positif. (3) $MAD \times 1,4826$ estime σ du bruit car le fond couvre presque toute l'image ; un z-score classique serait gonflé par les sticks brillants et effacerait le SNR absolu. Conséquence : un même seuil vaut pour toutes les scènes.

2.2 Les six cartes features()

#	carte	calcul	ce qu'elle apporte
1	z	image prétraitée telle quelle	position exacte du pixel (résolution sous-pixel conservée)
2	smooth $\sigma=1$	gaussien $\sigma=1$ px, puis re-blanchiment MAD	réduit le bruit blanc, SNR local du stick
3	smooth $\sigma=2$	gaussien $\sigma=2$ px, puis re-blanchiment MAD	contexte plus large, sticks faibles
4	matched	max sur 3 gabarits ($\theta = \pi/4$ et $\pi/4 \pm 0,2$), longueur 5 px, largeur 1 px, moyenne nulle, norme 1, par corrélation	filtre adapté à la forme attendue du stick
5	ridge	$-\lambda_{\min}$ de la Hessienne du lissé ($\sigma=1,2$), $\times \sigma^2$	détecte les structures étroites (crêtes)
6	écart-type local	$\sqrt{(\text{moyenne locale de } z^2 \text{ sur } 7 \times 7)}$	énergie locale

Le re-blanchiment (`_whiten` : soustraction de la médiane, division par $1,4826 \cdot \text{MAD}$ de la carte filtrée) remet chaque carte en unités de σ de *son* fond. Il est nécessaire après le max sur les gabarits, qui biaise le fond vers le haut.

2.3 Entraînement `LogReg.fit()`

```
F = features(z).reshape(-1, 6); y = m.reshape(-1) # un échantillon = un pixel
pos = np.flatnonzero(y) # tous les positifs
neg = rng.choice(np.flatnonzero(~y), 20 * len(pos), replace=False) # 20 négatifs par positif
X, t = F[idx], y[idx].astype(float)
self.mu, self.sd = X.mean(0), X.std(0) + 1e-6 # standardisation
X = np.c_[X - self.mu] / self.sd, np.ones(len(X))] # colonne de 1 = biais
w = np.zeros(7)
for i in range(25): # IRLS (Newton) pour la logistique
```

```

p = 1 / (1 + np.exp(-X @ w))
H = (X * (p*(1-p))[:, None]).T @ X + 1e-3*np.eye(7) # Hessienne + ridge 1e-3
w += np.linalg.solve(H, X.T @ (t - p))

```

Données : les 500 premières images de data/train, soit 11,250,000 pixels dont **38,819 positifs (0.35 %)**. Le sous-échantillonnage garde 38,819 positifs et 776,380 négatifs. **Optimisation** : IRLS (équivalent Newton), 25 itérations, régularisation L2 de 10^{-3} sur la Hessienne. Le fit prend 25 s, ce qui inclut le calcul des cartes sur les 500 images.

Pourquoi sous-échantillonner ? Les positifs ne font que 0,35 % des pixels ; sans cela, le classifieur prédirait « fond » partout. Prix à payer : les probabilités sont fausses par construction (le biais est décalé d'environ $\ln 20$). On ne les utilise jamais comme probabilités : on compare le **logit** à un seuil réglé ensuite sur val, ce qui absorbe ce décalage.

2.4 Inférence et choix du seuil

```

def scores(self, z):
    F = (features(z) - self.mu) / self.sd
    return F @ self.w[:-1] + self.w[-1] # logit par pixel
# seuil : balayage sur data/val, on garde celui qui maximise le tol-F1, puis il est gelé

```

Le seuil retenu est **2.5** (en logit). Il est choisi sur data/val uniquement, jamais sur le test ni sur l'OOD (piège de fuite de sélection). Le masque final est simplement `logit > seuil`, sans post-traitement morphologique.

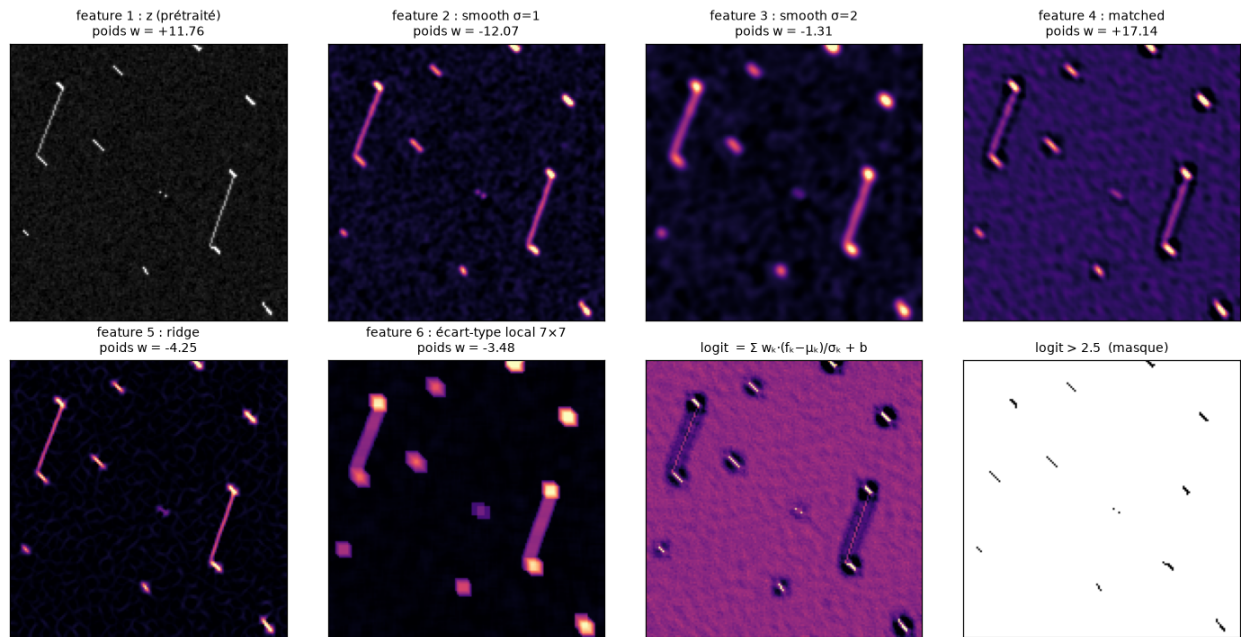
3. Paramètres appris

carte	μ	σ	poids w
z (prétraité)	1.01	5.20	+11.76
smooth $\sigma=1$	1.26	6.11	-12.07
smooth $\sigma=2$	1.27	5.52	-1.31
matched	1.03	5.80	+17.14
ridge	1.54	6.93	-4.25
écart-type local 7×7	1.57	2.18	-3.48
biais b			-7.61

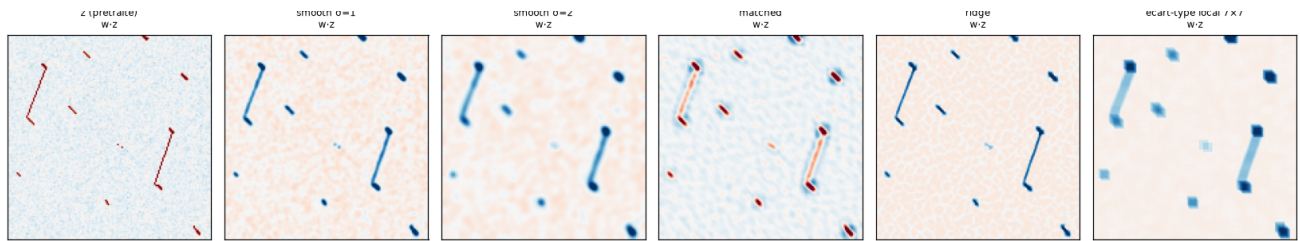
Les poids portent sur des features *standardisées*, donc leur amplitude est comparable. Lecture (mon interprétation, non démontrée par ablation) :

- **z (+11,8) et smooth $\sigma=1$ (-12,1)** : de signes opposés et presque égaux. La combinaison $\approx z - \text{lissé}(z)$ est un passe-haut : le modèle réclame un pixel qui ressort *par rapport à son voisinage*, ce qui localise le stick au pixel près. C'est probablement ce qui lui donne une IoU strict de 0,81, là où M1 à M4 plafonnent à 0,15–0,30.
- **matched (+17,1)** : le plus fort poids positif ; la forme attendue est le principal indice de présence.
- **ridge (-4,2) et écart-type local (-3,5)** : poids négatifs. Une ligne de charge est longue et fine, donc très « crête » et très énergétique sur 7×7 ; le modèle pénalise ces réponses, ce qui est cohérent avec la réduction des faux positifs sur les lignes de charge, que le masque officiel ne labellise pas.

4. Ce que voit le modèle sur une scène (test #172)



Les six cartes (haut et milieu), puis le logit et le masque après seuil. Le titre de chaque carte donne son poids.



Contribution w-z de chaque carte au logit (rouge : pousse vers « stick », bleu : vers « fond »).

5. Résultats

jeu	tol-F1	IoU strict	précision objet	rappel objet	F1 objet
val (seuil réglé ici)	0.930	0.816	0.902	0.926	0.914
test (400 scènes, seed 2025)	0.930	0.814	0.909	0.928	0.919
test + bruit supplémentaire (OOD)	0.913	0.780	0.923	0.871	0.897

rappel objet vs amplitude	[0,2)	[2,4)	[4,8)	[8,16)	[16,inf)
test	0.03 (n=174)	0.55 (n=283)	0.96 (n=895)	0.99 (n=1330)	0.99 (n=2501)
test + bruit	0.00	0.15	0.77	0.99	0.99

Coût : 41 ms par image sur CPU. Sous bruit supplémentaire, la baisse vient du rappel (0,93 → 0,87) : les sticks d'amplitude 2 à 8 sont perdus, tandis que la précision monte car le seuil figé devient plus sévère en pratique. Face à la meilleure méthode sans apprentissage (M2 : tol-F1 0,850, F1 objet 0,829), M5 gagne surtout sur les sticks faibles : rappel 0,55 contre 0,01 dans [2,4), et 0,96 contre 0,32 dans [4,8).

6. Limites et précautions

- **Apprise sur le masque du simulateur.** Ce masque est un seuil à 0,5 d'un rectangle flouté, fragmenté en environ 2,4 px par composante. M5 apprend ces artefacts de rasterisation, d'où l'IoU strict de 0,81 ; une partie de l'avance en IoU reproduit donc le générateur plutôt que la physique. Les métriques tolérantes et par objet sont plus fiables.

- **Dépendance au générateur.** μ , σ et les poids sont fixés pour ce simulateur (pente $\approx \pi/4$, niveau de bruit, pas de 2 mV). Un autre dispositif ou une autre résolution demanderait de réentraîner ; le test OOD du dépôt ne couvre que du bruit supplémentaire.

- **Faux positifs sur les lignes de charge.** Le modèle linéaire les réduit mais ne les supprime pas : sur la scène #150 il laisse des pointillés le long des lignes (précision objet 0,53 sur cette scène).
- **Pas d'ablation.** L'interprétation des poids (section 3) est une lecture, pas une mesure ; retirer une carte et réentraîner la confirmerait.
- **Poids opposés presque égaux (z et smooth $\sigma=1$).** Signe de colinéarité entre cartes : les poids isolés ne sont pas interprétables, seule leur combinaison l'est.